SUNV Smart Contract Deployment Guide

This guide covers the full deployment process for all 5 Sunvera smart contracts using a dual-chain architecture: Ethereum mainnet for the canonical token contract and treasury, Base L2 for user-facing operations.

Dual-Chain Architecture

Why Dual-Chain?

The dual-chain approach maximizes both institutional credibility and retail accessibility:

1. Ethereum mainnet = canonical token contract, treasury multisig, institutional trust anchor
2. Base L2 = user-facing operations (staking, governance voting, premium features, fee manager)
3. Arbitrum = secondary L2 for users who prefer that ecosystem (optional, Phase 3)

This gives you Ethereum L1 security guarantees for settlement and treasury, while users enjoy 2-second block times and ~$0.01 gas on Base.

Chain Responsibilities

Layer	Chain	Contracts	Purpose
L1 (Canonical)	Ethereum mainnet	SunveraToken (canonical), Treasury multisig	Token source of truth, institutional trust, treasury custody
L2 (Primary)	Base	SunveraToken (bridged), Staking, Governor, Timelock, FeeManager	User-facing operations, staking, governance, fees
L2 (Secondary)	Arbitrum	SunveraToken (bridged), Staking	Alternative L2 access (Phase 3)

Investor Appeal

Institutional investors see: 1. Canonical contract on Ethereum L1 — maximum security and settlement finality 2. Treasury secured by Gnosis Safe multisig on Ethereum mainnet 3. Coinbase-backed Base L2 — regulatory clarity and institutional infrastructure 4. EVM-compatible — integrates with Fireblocks, Copper, BitGo custody solutions

Retail investors see: 1. Gas costs ~$0.01 per transaction vs $5-50 on mainnet 2. 2-second block times vs 12 seconds on mainnet 3. Account abstraction (ERC-4337) — no ETH needed for gas (paymaster sponsored) 4. Accessible staking, voting, and premium features without prohibitive costs

---

Prerequisites

1. Wallet Setup

- Use a hardware wallet (Ledger or Trezor) as the deployer
- Fund the deployer wallet:
- Ethereum mainnet: 0.5-1 ETH for canonical contract deployment
- Base L2: 0.05-0.1 ETH for all user-facing contracts (much cheaper)
- Set up a Gnosis Safe (3-of-5 multisig) for each allocation wallet:
- Community & Ecosystem wallet (on Base)
- Team & Advisors wallet (on Ethereum)
- Treasury wallet (on Ethereum — institutional custody)
- Public Sale wallet (on Base)
- Private Sale wallet (on Ethereum)
- Liquidity wallet (on Base — for Uniswap V3 pool)
- Staking Rewards wallet (on Base)

2. Environment Configuration

Create a `.env` file in the project root:

```
# Network RPC
MAINNET_RPC_URL=https://eth-mainnet.g.alchemy.com/v2/YOUR_API_KEY
BASE_RPC_URL=https://mainnet.base.org
ARBITRUM_RPC_URL=https://arb1.arbitrum.io/rpc
SEPOLIA_RPC_URL=https://rpc.sepolia.org
BASE_SEPOLIA_RPC_URL=https://sepolia.base.org
```

```
# Deployer (hardware wallet via Ledger)
DEPLOYER_PRIVATE_KEY= # Leave empty for Ledger

# Etherscan API for verification
ETHERSCAN_API_KEY=YOUR_ETHERSCAN_API_KEY
BASESCAN_API_KEY=YOUR_BASESCAN_API_KEY

# Bridge configuration
WORMHOLE_RELAYER_URL= # For cross-chain messaging

# Optional: Alchemy for RPC
ALCHEMY_API_KEY=YOUR_ALCHEMY_KEY
```

3. Install Dependencies

```
npm install
npx hardhat compile
```

4. Network Selection

Network	Gas Cost (est.)	Block Time	Security	Use Case
Ethereum Mainnet	~$200-500	12 sec	Highest	Canonical token, treasury
Base L2	~$2-10	2 sec	High (Coinbase)	User-facing operations (PRIMARY)
Arbitrum	~$2-10	0.25 sec	High	Secondary L2 (Phase 3)
Sepolia (testnet)	Free	12 sec	Test only	L1 testnet
Base Sepolia (testnet)	Free	2 sec	Test only	L2 testnet (PRIMARY TEST)

Step-by-Step Deployment

Phase 1: Test on Base Sepolia (Mandatory)

Run the full deployment on Base Sepolia testnet first to verify everything works with L2 gas costs and speeds.

```
# Deploy all contracts to Base Sepolia
npx hardhat run scripts/deploy.js --network base_sepolia
npx hardhat run scripts/deployGovernance.js --network base_sepolia
npx hardhat run scripts/deployFeeManager.js --network base_sepolia
```

Verify on Basescan: 1. Token contract: check name, symbol, total supply, allocations 2. Governor contract: check voting delay, voting period, proposal threshold 3. Fee manager: check treasury, burn rates, staking integration 4. Test staking, governance proposal, and fee manager with small amounts 5. Test emergency pause and unpause 6. Test cross-chain bridge simulation (lock on testnet, mint on L2)

Also run on Ethereum Sepolia for the canonical token:

```
npx hardhat run scripts/deploy.js --network sepolia
```

Phase 2: Ethereum Mainnet — Canonical Token & Treasury

2a. Deploy the Canonical Token on Ethereum

```
npx hardhat run scripts/deploy.js --network mainnet
```

The deploy script will: 1. Deploy `SunveraToken` with all wallet addresses 2. Mint 100,000,000 SUNV to the contract 3. Distribute tokens to allocation wallets 4. Verify the contract on Etherscan 5. Output the canonical token contract address

Save this address — it is the permanent source of truth for SUNV.

2b. Set Up Treasury Multisig on Ethereum

1. Create a Gnosis Safe (3-of-5) on Ethereum mainnet
2. Transfer treasury allocation (15,000,000 SUNV) to the multisig
3. Add 5 signers (3 required for execution):
4. Technical lead
5. Legal counsel
6. Community representative
7. Security advisor
8. Founder/CEO
9. Configure spending limits and timelock policies on the multisig
10. Document the multisig address and signer roles

Phase 3: Base L2 — User-Facing Contracts

3a. Deploy Bridged Token on Base

Deploy a bridged version of the token on Base using Wormhole's Token Bridge:

```
# Deploy wrapped/bridged SUNV on Base
npx hardhat run scripts/deployBridgedToken.js --network base
```

This creates a bridged SUNV token on Base that is 1:1 backed by canonical SUNV locked on Ethereum.

3b. Deploy Staking on Base

```
npx hardhat run scripts/deploy.js --network base
```

This deploys `SunveraStaking` on Base, linked to the bridged token. Users stake on Base with ~$0.01 gas.

3c. Deploy Governance on Base

```
npx hardhat run scripts/deployGovernance.js --network base
```

This deploys: 1. `SunveraTimelock` (48-hour delay) on Base 2. `SunveraGovernor` on Base, linked to bridged token + timelock 3. Governance voting happens on Base (cheap, fast) 4. Proposals that affect Ethereum contracts are relayed via Wormhole messaging

3d. Deploy Fee Manager on Base

```
npx hardhat run scripts/deployFeeManager.js --network base
```

The FeeManager operates on Base for all platform fee collection, buyback-and-burn execution, and marketplace settlements.

Phase 4: Bridge Configuration

4a. Set Up Wormhole Bridge

1. Register SUNV with Wormhole Token Bridge on Ethereum mainnet
2. Register bridged SUNV on Base via Wormhole Portal

3. Configure bridge parameters:
4. Maximum transfer per transaction: 500,000 SUNV
5. Daily transfer limit: 5,000,000 SUNV
6. Large transfer timelock: 24 hours for transfers >1,000,000 SUNV
7. Test bridge with small amounts (100 SUNV) in both directions
8. Verify bridge contracts on both Etherscan and Basescan

4b. Liquidity Setup on Base

1. Create Uniswap V3 pool on Base (SUNV/ETH, 0.3% fee tier)
2. Add initial liquidity (5,000,000 SUNV + matching ETH from liquidity wallet)
3. Lock LP tokens via Team.Finance or UNCX (12-month minimum)
4. Verify pool is active and trading works
5. Record Uniswap pool address for fee manager buyback integration

Phase 5: Post-Deployment Configuration

5a. Fund the Staking Reward Pool on Base

```
npx hardhat console --network base
```

```
const token = await ethers.getContractAt("SunveraToken", "BRIDGED_TOKEN_BASE");
const staking = await ethers.getContractAt("SunveraStaking", "STAKING_BASE");

// Transfer bridged SUNV to staking contract
await token.connect(stakingWallet).transfer(staking.address, ethers.parseEther("5000000"));

// Fund the reward pool
await staking.connect(admin).fundRewardPool(ethers.parseEther("5000000"));
```

5b. Configure the Fee Manager on Base

```
const feeManager = await ethers.getContractAt("SunveraFeeManager", "FEE_MANAGER_BASE");

// Set staking contract address
await feeManager.connect(admin).setStakingContract(staking.address);

// Set Uniswap pool address for buyback
await feeManager.connect(admin).setUniswapPool(UNISWAP_POOL_BASE);

// Verify treasury (should be Ethereum multisig address)
console.log("Treasury:", await feeManager.treasury());
```

5c. Verify All Contracts

Ethereum mainnet:

```
npx hardhat verify --network mainnet TOKEN_ADDRESS_L1 --contract contracts/SunveraToken.sol
```

Base L2:

```
npx hardhat verify --network base TOKEN_ADDRESS_L2 --contract contracts/SunveraToken.sol:Su
npx hardhat verify --network base STAKING_ADDRESS --contract contracts/SunveraStaking.sol:S
npx hardhat verify --network base TIMELOCK_ADDRESS --contract contracts/SunveraTimelock.sol
npx hardhat verify --network base GOVERNOR_ADDRESS --contract contracts/SunveraGovernor.sol
npx hardhat verify --network base FEE_MANAGER_ADDRESS --contract contracts/SunveraFeeManage
```

Contract Addresses Record

After deployment, fill in these addresses and store securely:

```
=== ETCANONICAL (ETHEREUM MAINNET) ===
Network:           Ethereum Mainnet
Chain ID:          1
Deployment Date:   [YYYY-MM-DD]
Deployer Address:  [0x...]

SunveraToken (canonical): [0x...]
Treasury Multisig:        [0x...]

=== USER-FACING (BASE L2) ===
Network:           Base
Chain ID:          8453
Deployment Date:   [YYYY-MM-DD]
Deployer Address:  [0x...]

SunveraToken (bridged):  [0x...]
SunveraStaking:          [0x...]
SunveraTimelock:         [0x...]
SunveraGovernor:         [0x...]
SunveraFeeManager:       [0x...]
Uniswap V3 Pool:         [0x...]

=== BRIDGE ===
Wormhole Token Bridge (L1): [0x...]
Wormhole Token Bridge (L2): [0x...]
Bridge Transfer Limit:      500,000 SUNV/tx
Bridge Daily Limit:         5,000,000 SUNV/day
```

```
=== ALLOCATION WALLETS ===
Ethereum Mainnet:
  Team:            [0x...]
  Private Sale:    [0x...]
  Treasury:        [Gnosis Safe 3-of-5]

Base L2:
  Community:       [0x...]
  Public Sale:     [0x...]
  Liquidity:       [0x...]
  Staking:         [0x...]

=== SECURITY ===
LP Lock:           [Team.Finance / UNCX ID]
Bug Bounty:        https://immunefi.com/bounty/sunveracapital
```

Security Checklist

Before going live, complete ALL of these:

1. Security Audit: Hire CertiK, Hacken, or OpenZeppelin for a full audit of all 5 contracts
2. Bridge Audit: Separate audit of Wormhole bridge integration and transfer limits
3. Multisig Wallets: All allocation wallets are Gnosis Safe (3-of-5 minimum)
4. Timelock Verified: Governor cannot execute without 48-hour delay
5. Roles Verified: Deployer has renounced all roles (check on Etherscan and Basescan)
6. Contracts Verified: Source code verified on both Etherscan and Basescan
7. Liquidity Locked: LP tokens locked via Team.Finance or UNCX (12-month minimum)
8. Bug Bounty: Set up Immunefi bug bounty (minimum $10K) BEFORE mainnet launch
9. Testnet Complete: Full deployment tested on Base Sepolia AND Ethereum Sepolia
10. Admin Keys: Stored offline (hardware wallet + backup)
11. Bridge Limits: Transfer limits and timelock configured and tested
12. Recovery Plan: Document emergency pause, bridge pause, and recovery procedures
13. Formal Verification: Token and Staking contracts verified with Certora or Halmos
14. Forta Bots: Monitoring bots deployed for anomaly detection on both chains

Emergency Procedures

Pause All Contracts (Base)

```
// Pause token transfers on Base
await token.connect(admin).pause();

// Pause fee manager on Base
await feeManager.connect(admin).pause();
```

Pause Bridge (Ethereum)

```
// Pause Wormhole bridge to stop cross-chain transfers
await bridge.connect(admin).pauseTransfers();
```

Emergency Token Recovery

The fee manager can recover non-SUNV ERC20 tokens:

```
await feeManager.connect(admin).recoverERC20(tokenAddress, recipient, amount);
```

Governance Attack Response

1. Monitor for proposals from suspect addresses via Forta bot
2. If malicious proposal is submitted, rally community to vote AGAINST
3. The 48-hour timelock provides a window to respond
4. If proposal passes timelock, pause contracts as last resort
5. Engage legal counsel if criminal activity is involved

Gas Estimation

Ethereum Mainnet (Canonical Token Only)

Contract	Deployment Gas	Estimated Cost
SunveraToken (canonical)	~3,200,000	~$100-200

Contract	Deployment Gas	Estimated Cost
Treasury multisig setup	~500,000	~$15-30
Total L1	~3,700,000	~$115-230

Base L2 (User-Facing Contracts)

Contract	Deployment Gas	Estimated Cost
SunveraToken (bridged)	~2,000,000	~$1-5
SunveraStaking	~1,800,000	~$1-5
SunveraTimelock	~1,500,000	~$1-3
SunveraGovernor	~4,200,000	~$2-8
SunveraFeeManager	~2,800,000	~$1-5
Wormhole bridge setup	~1,000,000	~$1-3
Total L2	~13,300,000	~$7-29

Total dual-chain deployment: ~$122-260 (vs $450-780 for mainnet-only)

Account Abstraction (ERC-4337) Setup

To enable gasless transactions for users on Base:

1. Deploy a SUNV Paymaster contract on Base
2. Fund the paymaster with ETH from treasury (covers user gas costs)
3. Configure the paymaster to sponsor transactions for:
4. Staking and unstaking
5. Governance voting
6. Premium feature access checks
7. Set daily gas budget limit (e.g., 0.5 ETH/day)
8. Users interact via a smart account wallet (Biconomy, ZeroDev, or custom)

This removes the biggest UX barrier — users never need ETH to use SUNV features.

Post-Deployment Monitoring

1. Etherscan + Basescan: Set up alerts for all contract addresses on both chains
2. Dune Analytics: Create dashboards tracking:
3. Total supply over time (L1 + L2)
4. Bridge volume between Ethereum and Base
5. Burn rate across both chains
6. Staking participation on Base
7. Governance proposals and voter turnout
8. Forta Bots: Deploy monitoring bots for:
9. Unusual large transfers
10. Bridge anomaly detection
11. Governance attack patterns
12. Staking contract irregularities
13. Discord/Telegram: Set up contract event notifications for both chains
14. The Graph: Deploy subgraphs for instant UI queries on contract events

---

Legal & Compliance

Before public launch:

1. Legal Opinion: Obtain a securities law analysis (Howey test) for SUNV
2. KYC/AML: Integrate with Sumsub or Persona for token sale participants
3. OFAC Compliance: Geo-block restricted jurisdictions at the application level
4. Terms of Service: Update with token-specific terms and dual-chain disclosure
5. Risk Disclosure: Document all risks including bridge risks for token holders
6. Regulatory Filing: File necessary forms (Reg D, Reg S, etc.)
7. Bridge Disclosure: Inform users that bridged SUNV on Base is backed 1:1 by canonical SUNV locked on Ethereum

---

Support

For technical questions, open an issue on GitHub: https://github.com/hishamai8787-png/Sunvera-Capital-Holding-L.LC/issues

For security concerns, email: security@sunveracapital.com